

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Российский экономический университет им. Г.В. Плеханова»

Практическая работа №2

Поддержка и тестирование программных модулей
на тему: Создание приложения калькулятор

Специальность «Информационные системы и программирование»

Руководитель

_____ / Д.Б. Берестов/

Оценка _____

«_02_» _октября_ 2026 г.

Исполнитель

_____ / В.Д. Коноплев/

Группа ИПо-32

«_02_» _октября_ 2026 г.

Пермь, 2026

Оглавление

Введение	2
В данной практической работе я использовал	2
1. Создание решения	3
2. Создание библиотеки классов	4
3. Логика калькулятора	5
4. Создание WPF-приложения	6
5. Создание интерфейса калькулятора	7
6. Написание логики интерфейса	8
7. Создание проекта модульных тестов	11
8. Запуск тестов	14
Заключение	15

Введение

Visual Studio - интегрированная среда разработки для создания приложений. В практической работе разработано настольное приложение-калькулятор на языке C# с графическим интерфейсом WPF, отдельной библиотекой вычислений и модульными тестами.

В данной практической работе я использовал:

1. Среда разработки: Microsoft Visual Studio 2022/2026 и .NET SDK.
2. Технологии: WPF для интерфейса, .NET 8 для выполнения приложения, MSTest для автоматической проверки.
3. Язык: C# - объектно-ориентированный язык, на котором написаны вычисления и обработчики интерфейса.
4. Архитектура: решение разделено на библиотеку KonoplevLibraryFramework, приложение KonoplevWPF и тесты KonoplevTests.

Ход работы

1. Создание решения

Сначала было создано решение KonoplevCalc. Оно объединяет три проекта и позволяет независимо развивать вычислительную библиотеку, пользовательский интерфейс и набор модульных тестов.

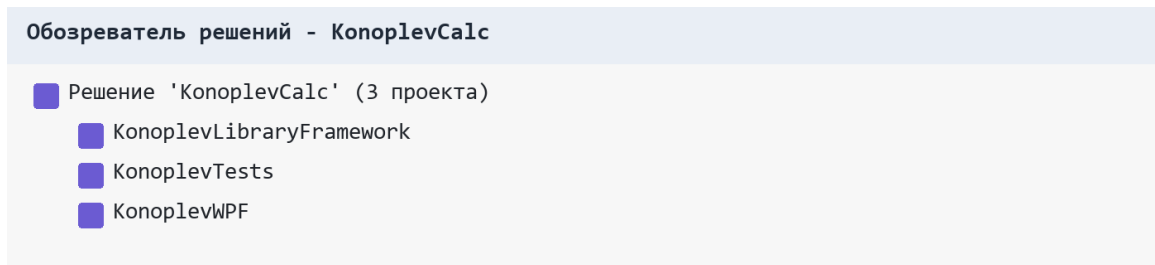


Рисунок 1 - Структура решения KonoplevCalc

Такое разделение упрощает сопровождение программы: интерфейс не содержит базовую арифметическую логику, а правильность библиотеки можно проверять без запуска окна.

2. Создание библиотеки классов

В решение добавлен проект KonoplevLibraryFramework. В нём создан класс KonoplevClass, содержащий методы выполнения арифметических операций.

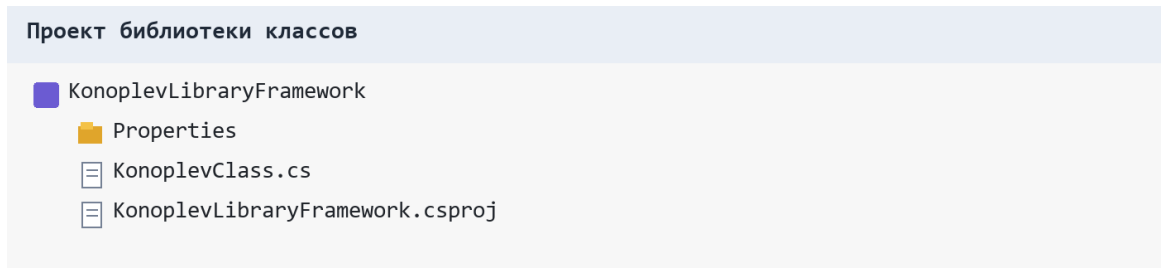


Рисунок 2 - Состав проекта библиотеки

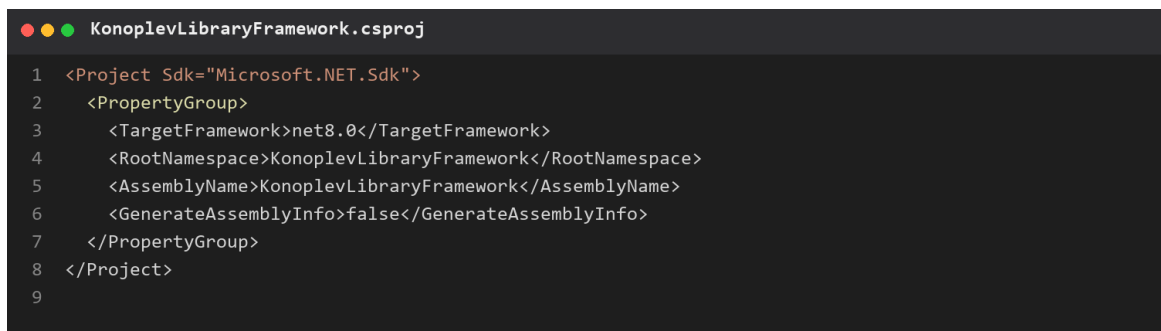
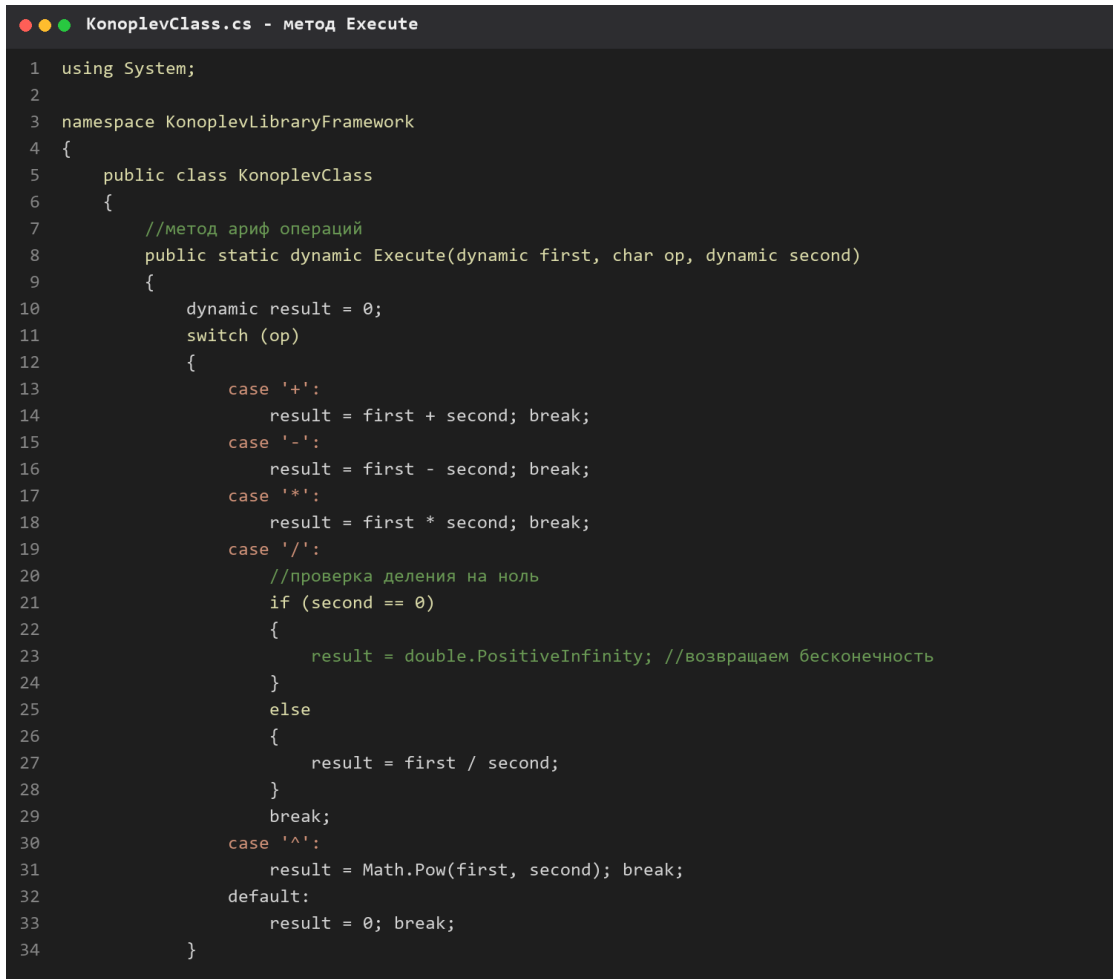


Рисунок 3 - Параметры проекта библиотеки

3. Логика калькулятора

Статический метод Execute принимает два числа и символ операции. Конструкция switch выполняет сложение, вычитание, умножение, деление или возведение в степень. При делении на ноль возвращается положительная бесконечность.

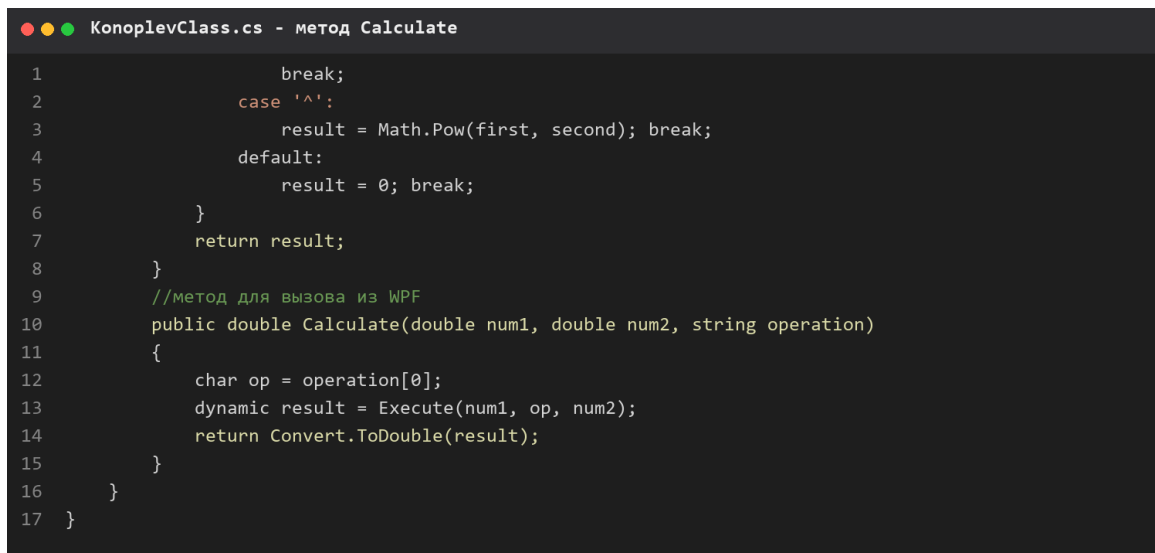


```
1  using System;
2
3  namespace KonoplevLibraryFramework
4  {
5      public class KonoplevClass
6      {
7          //метод ариф операций
8          public static dynamic Execute(dynamic first, char op, dynamic second)
9          {
10             dynamic result = 0;
11             switch (op)
12             {
13                 case '+':
14                     result = first + second; break;
15                 case '-':
16                     result = first - second; break;
17                 case '*':
18                     result = first * second; break;
19                 case '/':
20                     //проверка деления на ноль
21                     if (second == 0)
22                     {
23                         result = double.PositiveInfinity; //возвращаем бесконечность
24                     }
25                     else
26                     {
27                         result = first / second;
28                     }
29                     break;
30                 case '^':
31                     result = Math.Pow(first, second); break;
32             default:
33                 result = 0; break;
34             }
35         }
36     }
37 }
```

Рисунок 4 - Реализация метода Execute

Метод Calculate

Для удобного вызова из WPF-приложения добавлен экземплярный метод Calculate. Он преобразует строковое обозначение операции в символ, вызывает Execute и возвращает результат типа double.



```
1         break;
2         case '^':
3             result = Math.Pow(first, second); break;
4         default:
5             result = 0; break;
6     }
7     return result;
8 }
9 //метод для вызова из WPF
10 public double Calculate(double num1, double num2, string operation)
11 {
12     char op = operation[0];
13     dynamic result = Execute(num1, op, num2);
14     return Convert.ToDouble(result);
15 }
16 }
17 }
```

Рисунок 5 - Метод Calculate

Библиотека не зависит от интерфейса, поэтому может повторно использоваться в других приложениях и тестироваться отдельно.

4. Создание WPF-приложения

Для графического интерфейса создан проект KonoplevWPF. В него входят точка входа App.xaml, окно MainWindow и программная логика окна.



Рисунок 6 - Состав WPF-проекта

В проект добавлена ссылка на KonoplevLibraryFramework, поэтому методы библиотеки доступны обработчикам пользовательского интерфейса.

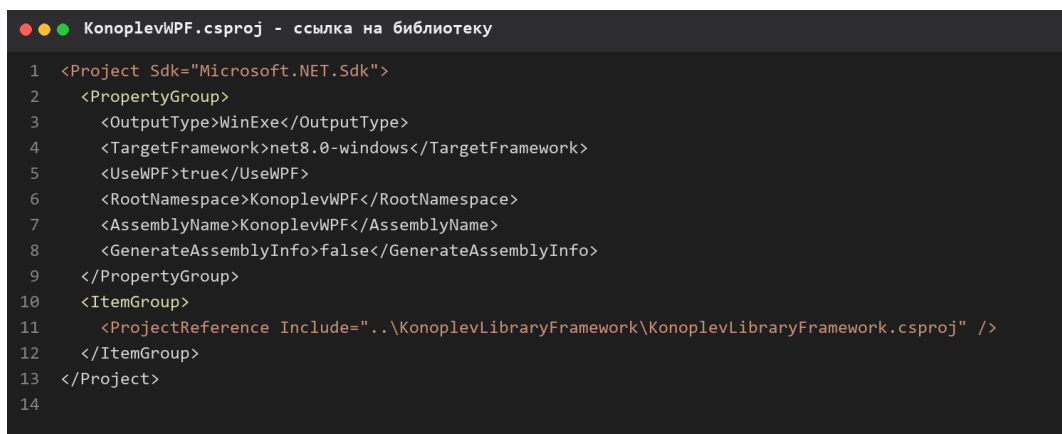


Рисунок 7 - Ссылка WPF-проекта на библиотеку

5. Создание интерфейса калькулятора

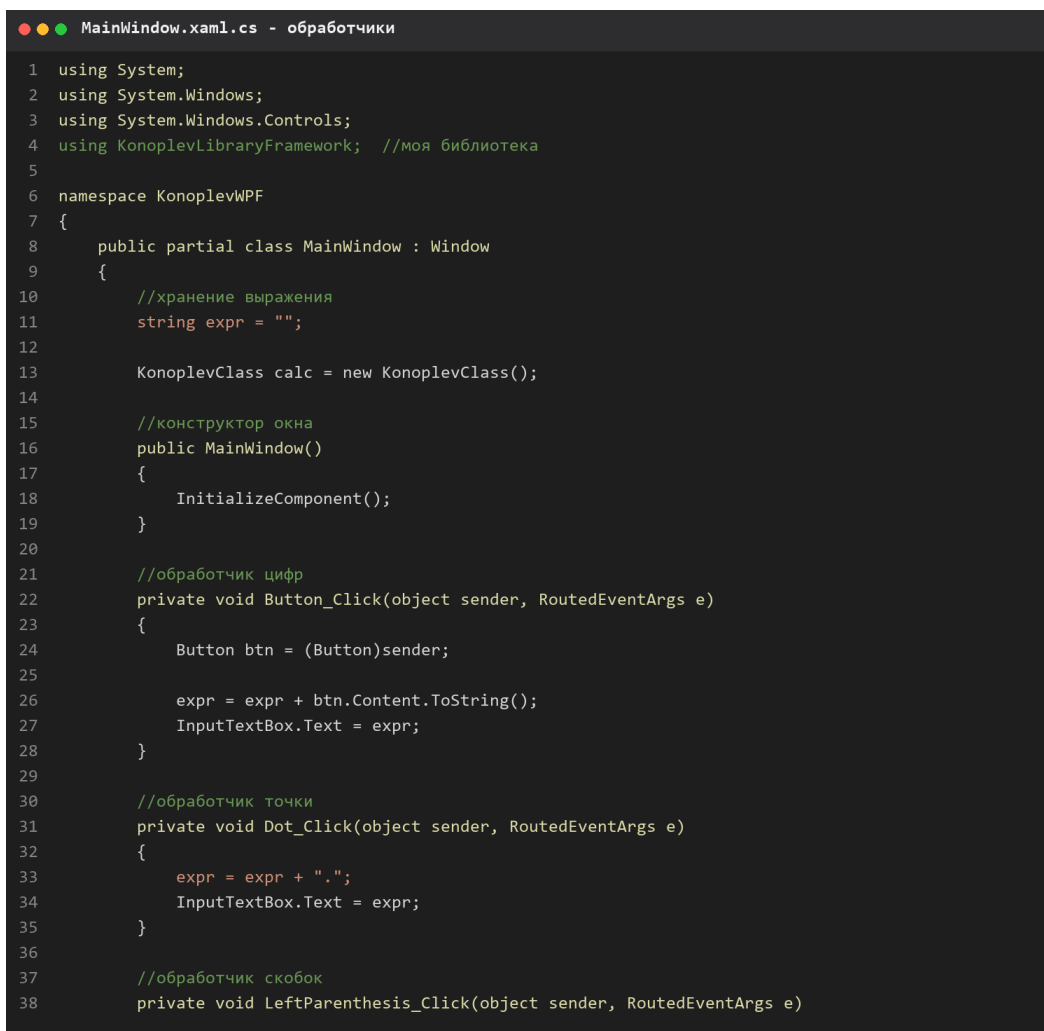
Главное окно содержит два поля вывода и сетку кнопок. Пользователь может вводить цифры, десятичную точку, скобки и операции $+$, $-$, $*$, $/$, $^$. Предусмотрены очистка, удаление последнего символа и вычисление результата.



Рисунок 8 - Интерфейс калькулятора

6. Написание логики интерфейса

В файле MainWindow.xaml.cs хранится текущее выражение и реализованы обработчики кнопок. При нажатии цифры или операции строка выражения обновляется и отображается пользователю.



```
1 using System;
2 using System.Windows;
3 using System.Windows.Controls;
4 using KonoplevLibraryFramework; //моя библиотека
5
6 namespace KonoplevWPF
7 {
8     public partial class MainWindow : Window
9     {
10         //хранение выражения
11         string expr = "";
12
13         KonoplevClass calc = new KonoplevClass();
14
15         //конструктор окна
16         public MainWindow()
17         {
18             InitializeComponent();
19         }
20
21         //обработчик цифр
22         private void Button_Click(object sender, RoutedEventArgs e)
23         {
24             Button btn = (Button)sender;
25
26             expr = expr + btn.Content.ToString();
27             InputTextBox.Text = expr;
28         }
29
30         //обработчик точки
31         private void Dot_Click(object sender, RoutedEventArgs e)
32         {
33             expr = expr + ".";
34             InputTextBox.Text = expr;
35         }
36
37         //обработчик скобок
38         private void LeftParenthesis_Click(object sender, RoutedEventArgs e)
```

Рисунок 9 - Обработчики событий MainWindow.xaml.cs

Разметка окна

Интерфейс описан декларативно в XAML. Сетка состоит из шести строк и четырёх столбцов, а обработчики Click связывают кнопки с программной логикой.

```
MainWindow.xaml - разметка
1 <Window x:Class="KonoplevWPF.MainWindow"
2     xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
3     xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
4     Title="Калькулятор Konoplev" Height="500" Width="350"
5     WindowStartupLocation="CenterScreen">
6
7     <Grid Margin="5">
8         <Grid.RowDefinitions>
9             <RowDefinition Height="50"/>
10            <RowDefinition Height="50"/>
11            <RowDefinition Height="*" />
12        </Grid.RowDefinitions>
13
14        <!-- поле ввода чисел и результата -->
15        <TextBox x:Name="InputTextBox"
16            Grid.Row="0"
17            FontSize="20"
18            IsReadOnly="True"
19            Background="White"
20            Text="0"/>
21
22        <!-- поле отображения вычисления -->
23        <TextBox x:Name="ResultTextBox"
24            Grid.Row="1"
25            FontSize="14"
26            IsReadOnly="True"
27            Background="LightGray"/>
28
29        <!-- сетка с кнопками -->
30        <Grid Grid.Row="2" Margin="0,5,0,0">
31            <Grid.RowDefinitions>
32                <RowDefinition Height="*" />
```

Рисунок 10 - Разметка MainWindow.xaml

Запуск и проверка калькулятора

После сборки решение было запущено. Для проверки выбрана операция возведения в степень: выражение 2^3 должно вернуть 8.

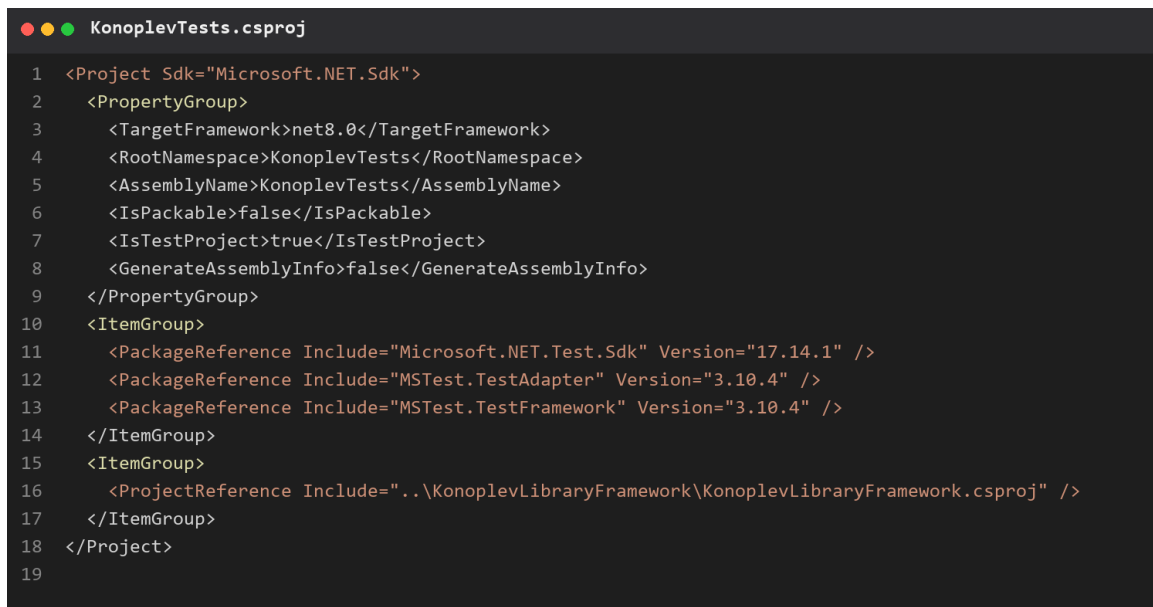


Рисунок 11 - Результат вычисления $2^3 = 8$

Результат получен корректно. Окно также поддерживает выражения со скобками, десятичные числа и приоритет арифметических операций.

7. Создание проекта модульных тестов

Для автоматической проверки создан проект KonoplevTests. Он использует фреймворк MSTest и содержит ссылку на тестируемую библиотеку.



```
1 <Project Sdk="Microsoft.NET.Sdk">
2   <PropertyGroup>
3     <TargetFramework>net8.0</TargetFramework>
4     <RootNamespace>KonoplevTests</RootNamespace>
5     <AssemblyName>KonoplevTests</AssemblyName>
6     <IsPackable>false</IsPackable>
7     <IsTestProject>true</IsTestProject>
8     <GenerateAssemblyInfo>>false</GenerateAssemblyInfo>
9   </PropertyGroup>
10  <ItemGroup>
11    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="17.14.1" />
12    <PackageReference Include="MSTest.TestAdapter" Version="3.10.4" />
13    <PackageReference Include="MSTest.TestFramework" Version="3.10.4" />
14  </ItemGroup>
15  <ItemGroup>
16    <ProjectReference Include="..\KonoplevLibraryFramework\KonoplevLibraryFramework.csproj" />
17  </ItemGroup>
18 </Project>
19
```

Рисунок 12 - Конфигурация проекта модульных тестов

Параметр IsTestProject сообщает среде сборки, что проект содержит тесты. Пакеты TestAdapter и TestFramework обеспечивают обнаружение и выполнение методов с атрибутом TestMethod.

Зависимости тестов

Проект тестов ссылается на библиотеку вычислений и пакеты MSTest. Благодаря этому тесты вызывают `KonoplevClass.Execute` и сравнивают фактические результаты с ожидаемыми.

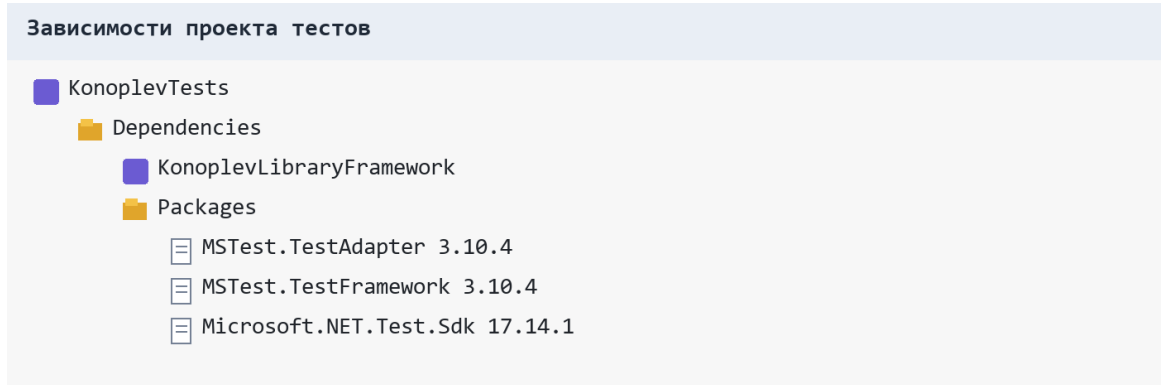


Рисунок 13 - Зависимости KonoplevTests

Всего подготовлено семь тестовых сценариев: пять основных операций, деление на ноль и вызов метода `Calculate`.

Код модульных тестов

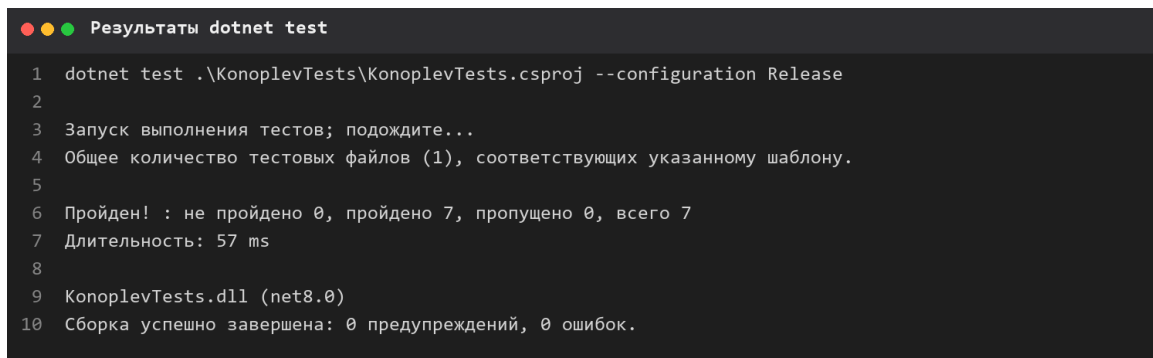
Каждый тест отмечен атрибутом [TestMethod], а класс - атрибутом [TestClass]. Проверка выполняется методом Assert.AreEqual.

```
KonoplevClassTests.cs
1  using Microsoft.VisualStudio.TestTools.UnitTesting;
2  using KonoplevLibraryFramework; // моя библиотека
3
4  namespace KonoplevTests
5  {
6      [TestClass]
7      public class KonoplevClassTests
8      {
9          //сложение
10         [TestMethod]
11         public void test_3plus3()
12         {
13             dynamic result = KonoplevClass.Execute(3, '+', 3);
14             Assert.AreEqual(6, result);
15         }
16
17         //вычитание
18         [TestMethod]
19         public void test_10minus4()
20         {
21             dynamic result = KonoplevClass.Execute(10, '-', 4);
22             Assert.AreEqual(6, result);
23         }
24
25         //умножение
26         [TestMethod]
27         public void test_6times7()
28         {
29             dynamic result = KonoplevClass.Execute(6, '*', 7);
30             Assert.AreEqual(42, result);
31         }
32
33         //деление
34         [TestMethod]
35         public void test_15divide3()
36         {
37             dynamic result = KonoplevClass.Execute(15, '/', 3);
38             Assert.AreEqual(5, result);
39         }
40
41         //степень
42         [TestMethod]
43         public void test_2to3()
44         {
45             dynamic result = KonoplevClass.Execute(2, '^', 3);
```

Рисунок 14 - Класс KonoplevClassTests

8. Запуск тестов

Тесты запущены командой `dotnet test` в конфигурации `Release`. Среда обнаружила один тестовый модуль и выполнила все семь проверок.



```
1 dotnet test .\KonoplevTests\KonoplevTests.csproj --configuration Release
2
3 Запуск выполнения тестов; подождите...
4 Общее количество тестовых файлов (1), соответствующих указанному шаблону.
5
6 Пройден! : не пройдено 0, пройдено 7, пропущено 0, всего 7
7 Длительность: 57 ms
8
9 KonoplevTests.dll (net8.0)
10 Сборка успешно завершена: 0 предупреждений, 0 ошибок.
```

Рисунок 15 - Результаты тестирования

Итог: не пройдено - 0, пройдено - 7, пропущено - 0. Сборка завершена без предупреждений и ошибок. Это подтверждает корректную работу `KonoplevLibraryFramework`.

Заключение

В ходе практической работы было разработано приложение «Калькулятор» с графическим интерфейсом WPF. Решение разделено на библиотеку вычислений, приложение и проект модульных тестов.

В процессе работы были закреплены навыки создания многопроектного решения, разработки интерфейса на XAML, обработки событий, реализации арифметических выражений, подключения проектных зависимостей и автоматического тестирования.

Приложение поддерживает сложение, вычитание, умножение, деление, возведение в степень и скобки. Для защиты от некорректного результата реализована обработка деления на ноль.

Все семь модульных тестов успешно пройдены. Проект собирается без ошибок и размещён в репозитории GitHub WhxRidd/KonoplevCalc. Поставленная задача выполнена полностью.